



Contexte

La galaxie est toujours sous la menace du Mulet. Vos équipages sont recensés, mais une simple liste ne suffit plus : la flotte **Galactica** compte cinq vaisseaux, des opérateurs aux métiers variés et trois **mentalistes** de la Seconde Fondation, capables d'influencer l'esprit des autres.

Il est temps de structurer votre programme avec des **objets**. Chaque personne, chaque vaisseau et la flotte elle-même deviennent des classes, qui portent leurs données **et** leurs comportements.

Ce TP en bref

- **Objectifs** : modéliser avec des classes, utiliser l'héritage, le polymorphisme, l'encapsulation et la composition, puis faire passer le programme du TP1 des dictionnaires aux objets.
- Il commence une fois le TP1 terminé. Vous avancez à votre rythme, en séance ou chez vous. Le projet complet est à rendre au plus tard le **jeudi 5 novembre à 16h30**.
- Il enrichit votre dépôt `4pyth-fondation-nom-prenom`. Commitez et poussez à chaque étape.
- L'IA est autorisée pour le projet, à une condition : **comprendre chaque ligne** de votre code.

Organisation des fichiers

Un fichier par domaine. Chaque fichier a une seule responsabilité. Revoyez au besoin le principe de responsabilité unique.

Fichier	Contenu
<code>person.py</code>	<code>ROLES</code> , <code>ACTIONS</code> et les classes <code>Person</code> , <code>Operator</code> et <code>Mentalist</code> (étape 1)
<code>spaceship.py</code>	la classe <code>Spaceship</code> (étape 2)
<code>fleet.py</code>	la classe <code>Fleet</code> (étape 2)
<code>test_classes.py</code>	les tests de vos classes (étape 3)
<code>fleet_data.py</code>	les données de la flotte, à copier (étape 4)
<code>crew.py</code>	les fonctions du menu, adaptées aux objets (étapes 4 et 5)
<code>main.py</code>	le menu interactif (étape 5)



Les personnes

Classes · héritage · @property

Dans le fichier `person.py`, écrivez trois **classes** : la classe mère `Person` et ses deux classes filles `Operator` et `Mentalist`.

La classe `Person` – une personne de la flotte

Élément	Rôle
<code>first_name</code> , <code>last_name</code> , <code>gender</code> , <code>age</code>	attributs initialisés par <code>__init__()</code>
<code>introduce_yourself()</code>	renvoie <code>Je m'appelle Bel Riose, je suis un homme de 48 ans.</code>
<code>__str__()</code>	renvoie <code>Bel Riose (M, 48 ans)</code>

Fallom et Joie n'ont pas de nom de famille

Pour eux, `last_name` vaut `""`. Vos affichages ne doivent pas laisser d'espace en trop après leur prénom. Pour retirer un membre de l'équipage, on utilisera son nom de famille... ou son prénom s'il n'en a pas.

La classe `Operator(Person)` – un membre d'équipage avec un rôle

Élément	Rôle
<code>role</code>	un rôle de la liste <code>ROLES</code> du TP1. Un rôle inconnu lève une <code>ValueError</code> . Validez-le avec un setter <code>@property</code>
<code>experience</code>	niveau d'expérience, <code>0</code> par défaut
<code>act()</code>	affiche l'action du membre selon son rôle (voir <code>ACTIONS</code> ↓) puis appelle <code>gain_experience()</code>
<code>gain_experience()</code>	augmente l'expérience de 1
<code>__str__()</code>	complète celui de <code>Person</code> avec son rôle : <code>Hugo Crast (M, 37 ans) – pilote</code>

```
ROLES = ["commandant", "pilote", "technicien", "armurier", "marchand", "entretien"]

ACTIONS = {
    "commandant": "donne ses ordres à l'équipage",
    "pilote": "pilote le vaisseau",
    "technicien": "répare les moteurs",
    "armurier": "vérifie les canons",
    "marchand": "négocie une cargaison",
    "entretien": "nettoie le vaisseau",
}
```

La classe `Mentalist(Person)` – un membre de la Seconde Fondation

Élément	Rôle
<code>mana</code>	points de mana, <code>100</code> par défaut, toujours compris entre 0 et 100 . Bornez-les avec un setter <code>@property</code>
<code>act()</code>	affiche <code>Joie se concentre.</code> et regagne 10 points de mana
<code>recharge_mana()</code>	regagne 50 points de mana
<code>influence(operator)</code>	coûte 20 points de mana et force l'opérateur à agir (appelle son <code>act()</code>). Sans assez de mana, affiche <code>Joie n'a pas assez de mana pour influencer Hugo Crast.</code> et renvoie <code>False</code>
<code>__str__()</code>	<code>Joie (F, 25 ans) – mentaliste, 100 mana</code>

Le polymorphisme en action

`Operator` et `Mentalist` ont tous deux une méthode `act()` **sans argument**. Un vaisseau pourra donc faire agir tout son équipage avec une simple boucle, sans jamais tester le type de chaque membre.

Indices

- Revoyez les notions d'héritage, de `super()` et de `@property` (validation d'une écriture).
- Dans `__init__()`, écrivez `self.role = role`. L'affectation passe par votre setter, donc la validation s'applique aussi à la création.
- Pour borner une valeur : `max(0, min(value, 100))`.
- Une propriété `name` qui assemble le prénom et le nom puis retire les espaces superflus (`strip()`) vous simplifiera tous les affichages.

2

Les vaisseaux et la flotte

Composition · attributs de classe · instance

Écrivez la classe `Spaceship` dans le fichier `spaceship.py` et la classe `Fleet` dans le fichier `fleet.py`. Importez les classes dont vous avez besoin, par exemple `from person import Operator`.

La classe `Spaceship` – un vaisseau et son équipage

Élément	Rôle
<code>name</code> , <code>ship_type</code> , <code>condition</code>	nom, type ("transport", "guerre" ou "marchand") et état ("opérationnel" par défaut ou "endommagé")
<code>crew</code>	liste d'objets <code>Operator</code> et <code>Mentalist</code> , vide au départ
<code>MAX_CREW = 10</code>	attribut de classe : capacité maximale
<code>add_member(member)</code>	ajoute un membre. Lève une <code>ValueError</code> si le vaisseau est plein ou si le nom est déjà à bord
<code>find_member(name)</code>	renvoie le membre correspondant ou <code>None</code>
<code>remove_member(name)</code>	retire le membre et le renvoie (ou <code>None</code> s'il est introuvable)
<code>check_preparation()</code>	renvoie <code>True</code> si l'équipage compte au moins un pilote et un technicien
<code>watch_round()</code>	tour de garde : chaque membre de l'équipage agit
<code>__str__()</code>	<code>Fearless (guerre, opérationnel) – 6 membres</code>

La classe `Fleet` – la flotte

Élément	Rôle
<code>name</code> , <code>spaceships</code>	nom de la flotte et liste d'objets <code>Spaceship</code> , vide au départ
<code>MAX_SPACESHIPS = 15</code>	attribut de classe : capacité maximale
<code>add_spaceship(spaceship)</code>	ajoute un vaisseau. Lève une <code>ValueError</code> si la flotte est pleine ou si le nom existe déjà
<code>find_spaceship(name)</code>	renvoie le vaisseau correspondant (sans tenir compte de la casse) ou <code>None</code>
<code>statistics()</code>	affiche le nombre de vaisseaux et de membres, la répartition par rôle, le nombre de mentalistes, l'expérience moyenne des opérateurs et le nombre de vaisseaux prêts

Indices

- Revoyez les notions de composition (des objets dans des objets) et d' `isinstance()` .
- Un mentaliste n'a pas de rôle. Dans `check_preparation()` et `statistics()` , ne regardez le rôle que des objets `Operator` .



3 Tester vos classes

Import • instantiation

Dans le fichier `test_classes.py`, importez vos classes, instanciez des objets de chaque classe et appelez leurs méthodes. Les classes restent ainsi séparées de leurs tests. Voici ce que doit afficher ce programme de test.

```

from person import Operator, Mentalist
from spaceship import Spaceship

bel = Operator("Bel", "Riose", "M", 48, "commandant")
hugo = Operator("Hugo", "Crast", "M", 37, "pilote")
joie = Mentalist("Joie", "", "F", 25)

print(bel.introduce_yourself())    # Je m'appelle Bel Riose, je suis un hom
me de 48 ans.
print(joie.introduce_yourself())  # Je m'appelle Joie, je suis une femme d
e 25 ans.
print(hugo)                       # Hugo Crast (M, 37 ans) – pilote
print(joie)                       # Joie (F, 25 ans) – mentaliste, 100 man
a

hugo.act()                        # Hugo Crast pilote le vaisseau.
print(hugo.experience)           # 1

for _ in range(6):
    joie.influence(hugo)         # 5 fois : Joie influence Hugo Crast. (p
uis Hugo agit)

    # 6e fois : Joie n'a pas assez de mana...
    # 0

print(joie.mana)                 # 0
joie.recharge_mana()
joie.recharge_mana()
joie.recharge_mana()
print(joie.mana)                 # 100 : le mana ne dépasse jamais 100

ship = Spaceship("Fearless", "guerre")
ship.add_member(bel)
ship.add_member(hugo)
print(ship.check_preparation())  # False : pas de technicien
ship.add_member(Operator("Yugo", "Amary", "M", 33, "technicien"))
print(ship.check_preparation())  # True
print(ship)                      # Fearless (guerre, opérationnel) – 3 me
mbres

Operator("Eve", "Hacker", "F", 30, "root")
# ValueError: rôle inconnu : root

```

4

Construire la flotte

Dictionnaires · objets

Copiez les données de la flotte Galactica dans le fichier `fleet_data.py`. Chaque vaisseau et chaque membre y est décrit par un **dictionnaire**. La clé `type` indique s'il faut créer un `Operator` ou un `Mentalist`.

Données à copier – `fleet_data.py` (5 vaisseaux, 24 membres)

```

FLEET_DATA = {
    "name": "Galactica",
    "spaceships": [
        {"name": "Bayta", "ship_type": "marchand", "condition": "opérationnel", "crew": [
            {"type": "operator", "first_name": "Hugo", "last_name": "Crast", "gender": "M", "age": 37, "role": "pilote"},
            {"type": "operator", "first_name": "Gaal", "last_name": "Dornick", "gender": "F", "age": 34, "role": "technicie
n"},
            {"type": "operator", "first_name": "Lathan", "last_name": "Devers", "gender": "M", "age": 46, "role": "marchand"},
            {"type": "operator", "first_name": "Novi", "last_name": "Sura", "gender": "F", "age": 25, "role": "entretien"},
        ]},
        {"name": "Dark Nebula", "ship_type": "guerre", "condition": "endommagé", "crew": [
            {"type": "operator", "first_name": "Jagger", "last_name": "Fount", "gender": "M", "age": 41, "role": "pilote"},
            {"type": "operator", "first_name": "Munn", "last_name": "Compot", "gender": "M", "age": 27, "role": "technicien"},
            {"type": "operator", "first_name": "Salvor", "last_name": "Hardin", "gender": "F", "age": 28, "role": "armurier"},
            {"type": "operator", "first_name": "Ebling", "last_name": "Mis", "gender": "M", "age": 34, "role": "entretien"},
            {"type": "mentalist", "first_name": "Tellem", "last_name": "Bond", "gender": "M", "age": 48, "mana": 100},
        ]},
        {"name": "Fearless", "ship_type": "guerre", "condition": "opérationnel", "crew": [
            {"type": "operator", "first_name": "Bel", "last_name": "Riose", "gender": "M", "age": 48, "role": "commandant"},
            {"type": "operator", "first_name": "Hober", "last_name": "Mallow", "gender": "M", "age": 52, "role": "pilote"},
            {"type": "operator", "first_name": "Yugo", "last_name": "Amary", "gender": "M", "age": 33, "role": "technicien"},
            {"type": "operator", "first_name": "Han", "last_name": "Pritcher", "gender": "M", "age": 31, "role": "armurier"},
            {"type": "operator", "first_name": "Harlan", "last_name": "Branno", "gender": "F", "age": 38, "role": "entretie
n"},
            {"type": "mentalist", "first_name": "Yanna", "last_name": "Seldon", "gender": "F", "age": 52, "mana": 100},
        ]},
        {"name": "Unimara", "ship_type": "marchand", "condition": "opérationnel", "crew": [
            {"type": "operator", "first_name": "Fallom", "last_name": "", "gender": "F", "age": 28, "role": "pilote"},
            {"type": "operator", "first_name": "Toran", "last_name": "Darell", "gender": "M", "age": 52, "role": "technicie
n"},
            {"type": "operator", "first_name": "Limmar", "last_name": "Ponyets", "gender": "M", "age": 43, "role": "marchan
d"},
            {"type": "operator", "first_name": "Jord", "last_name": "Commasson", "gender": "M", "age": 53, "role": "entretie
n"},
            {"type": "mentalist", "first_name": "Joie", "last_name": "", "gender": "F", "age": 25, "mana": 100},
        ]},
        {"name": "Weinis", "ship_type": "marchand", "condition": "endommagé", "crew": [
            {"type": "operator", "first_name": "Lewis", "last_name": "Pirenne", "gender": "M", "age": 34, "role": "pilote"},
            {"type": "operator", "first_name": "Stora", "last_name": "Gendibal", "gender": "F", "age": 35, "role": "technicie
n"},
            {"type": "operator", "first_name": "Eskel", "last_name": "Gorov", "gender": "M", "age": 32, "role": "marchand"},
            {"type": "operator", "first_name": "Bayta", "last_name": "Darell", "gender": "F", "age": 48, "role": "entretien"},
        ]},
    ],
}

```

Dans le fichier `crew.py`, écrivez la fonction `build_fleet(data)`, qui parcourt ces dictionnaires et renvoie un objet `Fleet` complet, avec ses vaisseaux et leurs équipages.

Gardez ce format en tête

Au TP3, vous sauvegarderez la flotte dans un fichier JSON... **exactement dans ce format.**

`build_fleet()` vous servira alors à la recharger.



Le menu en objets

Menu • exceptions

1. Au lancement, `main.py` construit la flotte avec `build_fleet(FLEET_DATA)`.
2. Adaptez les options [1] à [4] du TP1. On choisit d'abord le vaisseau **par son nom**, puis on agit sur son équipage. Pour ajouter un membre, on précise s'il s'agit d'un opérateur ou d'un mentaliste.
3. Ajoutez les options **[5] Ajouter un vaisseau** et **[6] Statistiques de la flotte**. Le TP3 ajoutera [7] et [8].
4. Les `ValueError` levées par vos classes (vaisseau plein, nom déjà pris, rôle inconnu...) sont interceptées et affichées. **Elles ne font jamais planter le programme.**

```
Proposition d'affichage

===== Flotte marchande – Poste de commandement =====
[1] Ajouter un membre
[2] Retirer un membre
[3] Afficher la flotte
[4] Vérifier un vaisseau
[5] Ajouter un vaisseau
[6] Statistiques de la flotte
[0] Quitter
Votre choix : 6
Flotte Galactica : 5 vaisseaux, 24 membres
  pilote : 5
  technicien : 5
  marchand : 3
  entretien : 5
  armurier : 2
  commandant : 1
  mentaliste : 3
Expérience moyenne des opérateurs : 0.0
Vaisseaux prêts : 5 / 5
```

Commitez !

Un commit par étape terminée: `git add .` puis `git commit -m "Ajout des classes Spaceship et Fleet"` et `git push`.



Pour aller plus loin

Facultatif

Bonus 1 – Un équipage vraiment prêt

- `check_preparation()` exige aussi un **mentaliste ayant au moins 50 points de mana**.
- Un vaisseau "endommagé" n'est jamais prêt. Combien de vaisseaux de la flotte restent prêts ?

Bonus 2 – Nouvelles options du menu

- **Tour de garde** : tout l'équipage d'un vaisseau agit (`watch_round()`).
- **Influence** : un mentaliste influence un opérateur de son vaisseau.
- Choisissez des numéros libres. Les options [7] et [8] sont réservées au TP3.

Bonus 3 – Promotions

- Comme au TP1, un technicien ayant au moins 10 points d'expérience peut être **promu pilote**. Cette fois, c'est une méthode `promote()` de la classe `Operator`.

Bonus 4 – Messages immersifs

- Enrichissez les messages de vos classes pour faire vivre l'univers de Fondation : alertes, dialogues, ambiance...



Avant de continuer

Vérifiez chaque point. Ils correspondent à la grille d'évaluation du projet Fondation.

- ☐ Les cinq classes existent avec leurs attributs et leurs méthodes. Vos tests donnent les résultats attendus.
- ☐ Chaque domaine a son fichier : `person.py` , `spaceship.py` , `fleet.py` , `test_classes.py` , `crew.py` et `main.py` .
- ☐ `Operator` et `Mentalist` héritent de `Person` et appellent `super().__init__()` .
- ☐ Un rôle inconnu est refusé et le mana reste toujours entre 0 et 100.
- ☐ Les options [0] à [6] du menu fonctionnent et aucune erreur ne fait planter le programme.
- ☐ Les dictionnaires du TP1 ont disparu. Le menu ne manipule plus que des objets.
- ☐ Chaque classe et chaque méthode a une docstring. Les noms sont en `snake_case` (classes en `PascalCase`).
- ☐ Plusieurs commits ont été poussés sur GitHub depuis le TP1.
- ☐ Vous pouvez expliquer **chaque ligne** de votre code.